

Simulation Tools

Shelcia Samuvel – 153055122

Ashinsani Rajapaksha - 153050017

Simulation Overview

- Its primary purpose is to validate whether the design being created functions according to the specified requirements.
- By running simulations early in the design phase, potential issues can be identified, thus minimizing the need for extensive code revisions.
- Simulations can be performed at different levels of abstraction and at various stages throughout the design process.

Simulation Overview ctd..

- From an abstraction perspective, simulations can be performed at Register-transfer level (RTL) and at gate level.
- Gate-level simulation can further be subdivided as timing and zero-delay simulations depending on whether timing information is considered during the simulation process

UVM Simulation Tools

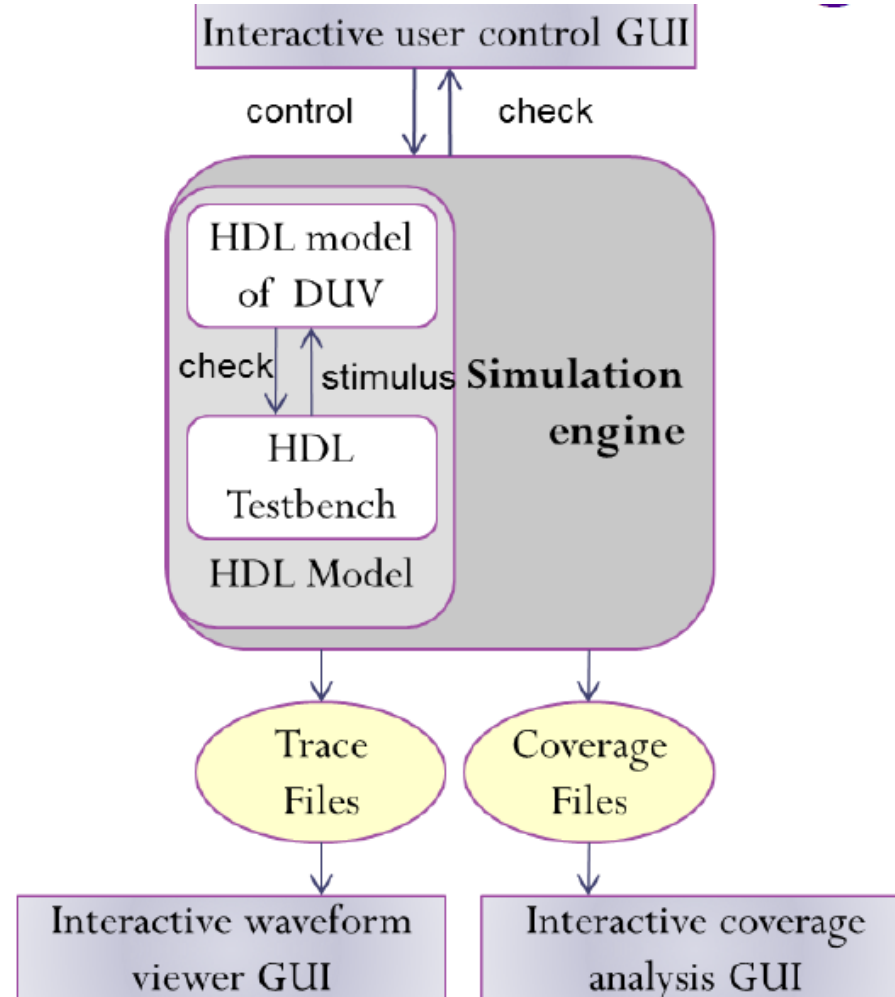
UVM simulation tools help verify complex designs by:

- Automating stimulus generation (UVM Sequences)
- Debugging testbenches (Transaction logging & waveform analysis)
- Ensuring verification completeness (Functional & code coverage)

Why is it important?

- Reduces testbench complexity.
- Improves reusability and scalability.
- Enhances Verification Efficiency

Basic HDL Simulator Block Diagram



1. Event-Driven Simulation Engines

Event-driven simulation engines update the circuit state only when an event occurs. These simulators are efficient for designs with limited switching activity.

Examples:

- Synopsys VCS
- Cadence Xcelium
- Siemens Questa
- Modelsim

Use Case:

Best suited for verifying **complex digital circuits** with conditional operations, such as control logic and finite state machines (FSMs).

2. Cycle-based Simulation Engines

Cycle-based simulators evaluate the entire circuit at each clock cycle, rather than tracking individual events. This makes them significantly faster for synchronous designs with high switching activity.

Examples:

- Synopsys ZeBu
- Cadence Palladium
- Siemens Veloce
- BlueSpec Bluesim

Use Case:

Ideal for datapath-heavy designs, such as DSP blocks, ALUs, and CPUs, where operations are evaluated at the clock boundaries.

Common Simulation Tools

We are going to focus on the following tools in this presentation

1. Synopsys VCS
2. Siemens Questa
3. Cadence Xcelium

The Synopsys logo, featuring the word "SYNOPSYS" in a bold, purple, sans-serif font with a registered trademark symbol.The Siemens logo, featuring the word "SIEMENS" in a bold, teal, sans-serif font.The Cadence logo, featuring the word "cādence" in a black, lowercase, sans-serif font with a red horizontal bar over the 'a' and a registered trademark symbol.

Feature Comparison

Feature	Synopsys VCS	Siemens Questa (ModelSim)	Cadence Xcelium
Language Support	SystemVerilog, Verilog, VHDL	SystemVerilog, Verilog, VHDL	SystemVerilog, Verilog, VHDL
Performance	High-speed event-driven & cycle-based optimizations	Optimized for debugging & ease of use	Performance-focused with parallelization
Waveform Debugging	FSDB format (Verdi)	WLF format (Waveform viewer)	SHM format (SimVision)
Coverage Analysis	Functional & Code Coverage, UPF	Functional & Code Coverage	Functional & Code Coverage, Low-Power Analysis
Parallel Simulation	Multithreading, Parallel builds	Multithreading	Multithreading & Hardware acceleration
Assertion Support	SVA, PSL	SVA, PSL	SVA, PSL
Integration	Verdi for Debugging, Low-Power Simulation	Visualizer GUI for Debugging	SimVision for Debugging

Core Mechanism of Event-Driven Simulation

Mechanism	Description
Event Detection	Detects signal value changes and triggers updates.
Event Queue	Maintains scheduled signal updates in timestamp order.
Delta Cycles	Handles zero-time ordering of updates within a single time step.
Signal Update Rules	Manages blocking vs non-blocking assignments and dependencies.
Time Advancement	Steps through events in chronological order.
Process Execution	Executes only affected processes to optimize performance.

Event Detection and Scheduling

- The simulator continuously monitors signals for events (value changes).
- When an event occurs, the simulator schedules the affected processes for execution.
- Only processes with an event on their sensitivity list are triggered, avoiding unnecessary computations.
- **Example (Verilog Sensitivity List):**

```
A1 @(posedge clk)
```

```
    q <= d;    // Executes only on clk rising edge
```

Event Queue (Scheduling and Execution Order)

- The simulator maintains an event queue, which stores all pending signal updates.
- Events are processed in timestamp order to ensure accurate simulation behavior.
- Events scheduled for the same timestamp are ordered using delta cycles (explained next).
- Event Queue Structure:

Simulation Time	Scheduled Events
10 ns	clk ↑, reset ↑
10 ns + $\delta 1$	q1 = d ($\delta 1$)
10 ns + $\delta 2$	q2 = q1 ($\delta 2$)

Delta Cycles (Zero-Time Steps for Ordering Updates)

- Delta delays (δ) ensure that updates happen in a specific order without advancing time.
- Why needed? To prevent race conditions in combinational logic.
- Rule: Within the same time step, assignments execute in delta cycle order.
- **Example (VHDL Delta Delay Mechanism):**

```
process(clk)
begin
    if rising_edge(clk) then
        q1 <= d;  -- Executes in  $\delta_1$ 
        q2 <= q1; -- Executes in  $\delta_2$  (q2 gets old q1)
    end if;
end process;
```

Signal Update Mechanism

- Blocking (=) vs Non-blocking (<=) Assignments in Verilog
 - Blocking (=): Executes immediately.
 - Non-blocking (<=): Schedules updates using delta delays.
- **Example:**
 - A1 @(posedge clk) begin
 - q1 = d; // Blocking
 - q2 = q1; // Gets new q1 immediately
 - end
 - A1 @(posedge clk) begin
 - q1 <= d; // Scheduled in δ_1
 - q2 <= q1; // Scheduled in δ_2 , gets old q1
 - end

Time Advancement (Simulation Kernel Execution)

- The event-driven simulation kernel follows these steps:
 - Find the next event in the event queue (smallest timestamp).
 - Process all events scheduled for that time.
 - Execute affected processes and update signals.
 - Schedule any resulting signal changes for execution in future delta cycles or time steps.
 - Repeat until no more events remain.

Efficient Process Execution (Sensitivity List & Hierarchical Scheduling)

- Each process runs only if an event on its sensitivity list occurs.
- The simulator avoids unnecessary computations, making event-driven simulation highly scalable.
- **Example (VHDL Process with Sensitivity List):**

```
process(clk, reset)
begin
    if reset = '1' then
        q <= '0';
    else if rising_edge(clk) then
        q <= d;
    end if;
```


Summary

- Event-driven simulation processes updates only when an event occurs, making it efficient for complex digital circuits.
- Cycle-based simulation evaluates the entire circuit at each clock cycle, making it faster for synchronous designs.
- Event-driven simulation workflow includes event detection, event queue processing, delta cycles, and signal updates.
- Delta delays (δ) ensure proper ordering of signal updates within the same time step.
- Signal update mechanisms involve blocking (=) vs. non-blocking (<=) assignments.
- Time advancement and process execution optimize simulation efficiency.

References

- Lecture 9: Simulation Engines, Logic Synthesis Course, Tampere University
- <https://www.synopsys.com/content/dam/synopsys/gated-assets/verification/vcs-ds.pdf>
- <https://static.sw.cdn.siemens.com/siemens-disw-assets/public/QzFgMxW5gizEDRIAZYTQE/en-US/Siemens-SW-QuestaSim-FS-85329-D5.pdf>
- https://www.cadence.com/en_US/home/tools/system-design-and-verification/simulation-and-testbench-verification/xcelium-simulator.html

Thank You!

Questions

1. Compare Event-Driven Simulation vs. Cycle-Based Simulation Engines.
2. Explain the core mechanisms of Event-Driven Simulation.